

NAME:- GAURAV SURESH GIRI

PRN :- 202501050012

ROLL NO :- 12

BATCH :- CH1

DIVISION:- CH(CHEMICAL)

1.1.1. Calculate Momentum

02:20



Write a program that accepts the mass of an object (in kilograms) and its velocity (in meters per second), then calculates and displays the momentum of the object. The momentum p is calculated using the formula:

$$p = m \times v$$

where:

m is the mass of the object (in kilograms).

v is the velocity of the object (in meters per second).

Input Format:

- A single floating-point number representing the mass of the object in kilograms.
- A single floating-point number representing the velocity of the object in meters per second.

Output Format:

- The output will display calculated momentum with appropriate units (kgm/s) (rounded up to 2 decimal places).

Sample Test Cases

Test case 1

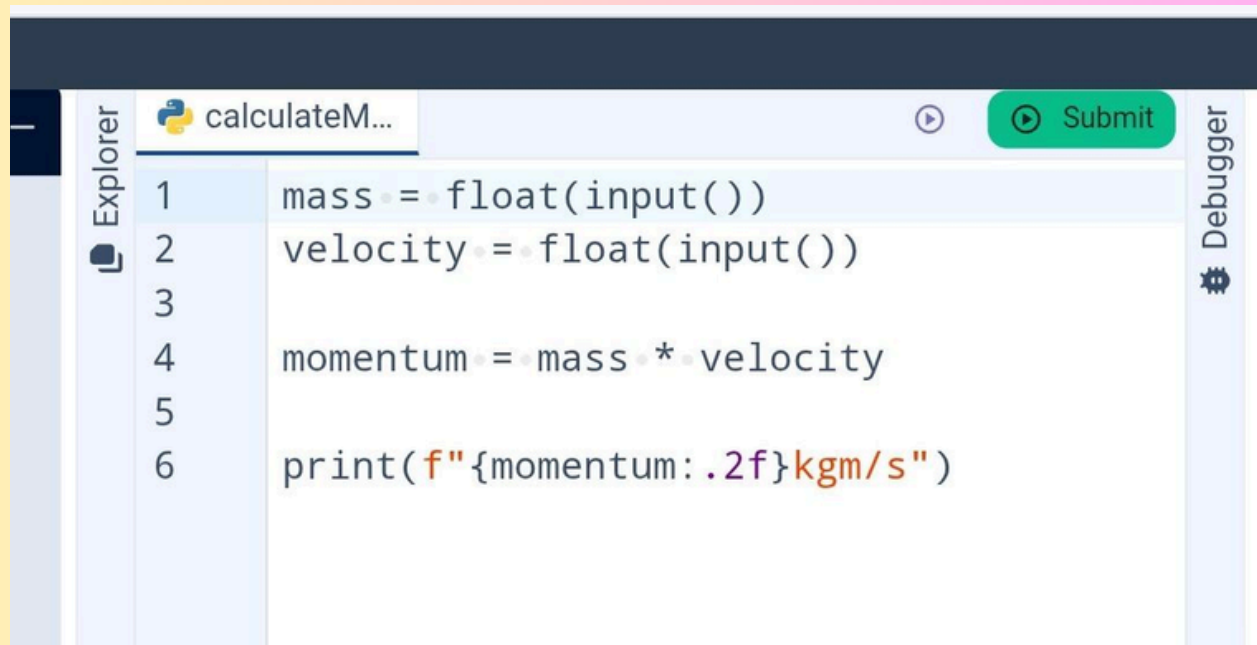
5.0

10.0

50.00kgm/s

Test case 2

Input:

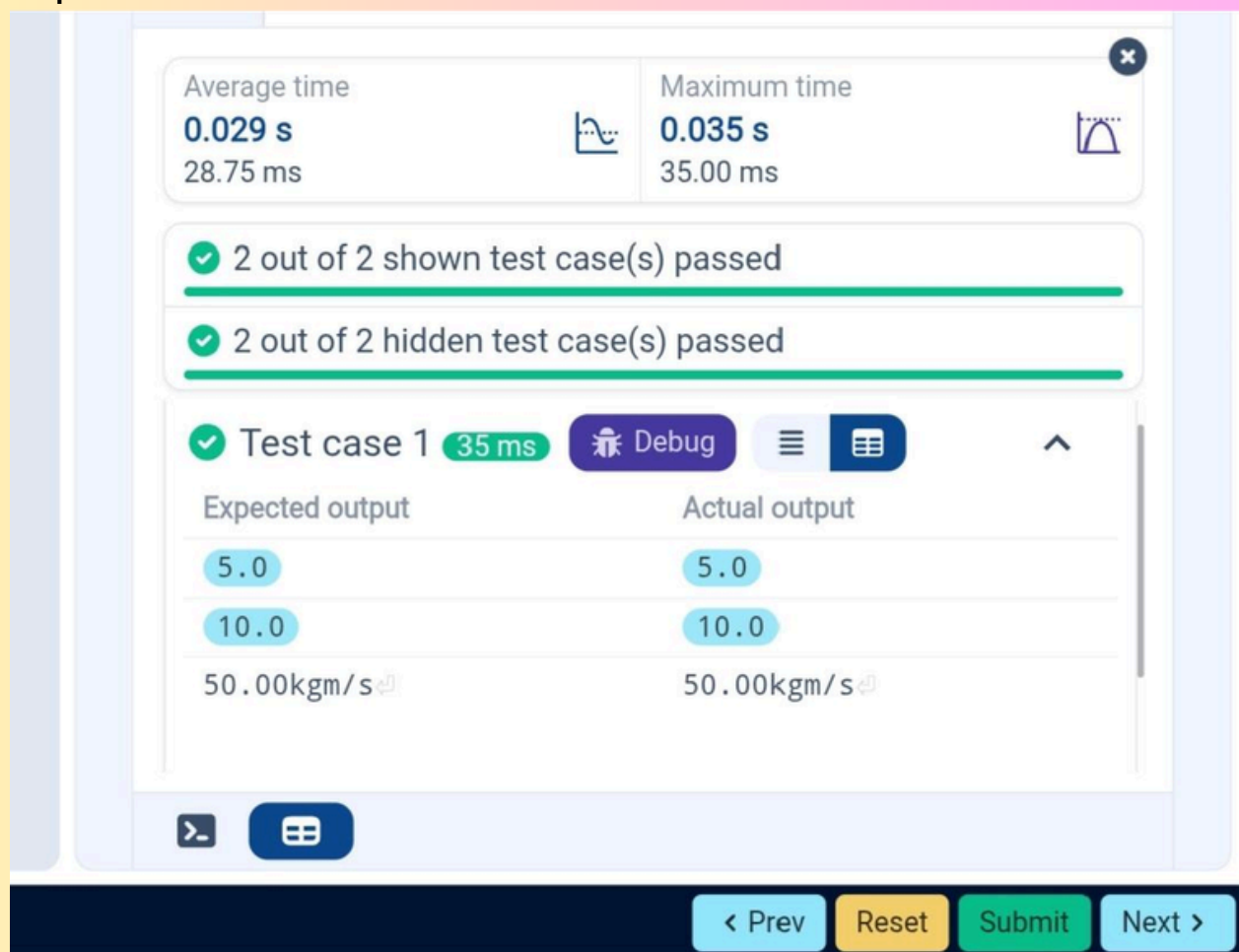


The screenshot shows a code editor with a file named 'calculateM...'. The code is as follows:

```
1 mass = float(input())
2 velocity = float(input())
3
4 momentum = mass * velocity
5
6 print(f"{momentum:.2f}kgm/s")
```

On the left side, there is an 'Explorer' panel with a folder icon. On the right side, there is a 'Debugger' panel with a bug icon. At the top right, there is a green 'Submit' button.

Output:



The screenshot displays a test results interface. At the top, there are two boxes for performance metrics:

- Average time:** 0.029 s (28.75 ms)
- Maximum time:** 0.035 s (35.00 ms)

Below these, there are two green checkmarks indicating that all test cases passed:

- 2 out of 2 shown test case(s) passed
- 2 out of 2 hidden test case(s) passed

The 'Test case 1' section is expanded, showing a duration of 35 ms and a 'Debug' button. It contains a table comparing 'Expected output' and 'Actual output':

Expected output	Actual output
5.0	5.0
10.0	10.0
50.00kgm/s	50.00kgm/s

At the bottom of the interface, there are navigation buttons: '< Prev', 'Reset', 'Submit', and 'Next >'.

Write a Python program that accepts an integer n as input.
Depending on the number of digits in n .

Constraints:

$$1 \leq n \leq 999$$

 Input Format:

- The input consists of a single integer n .

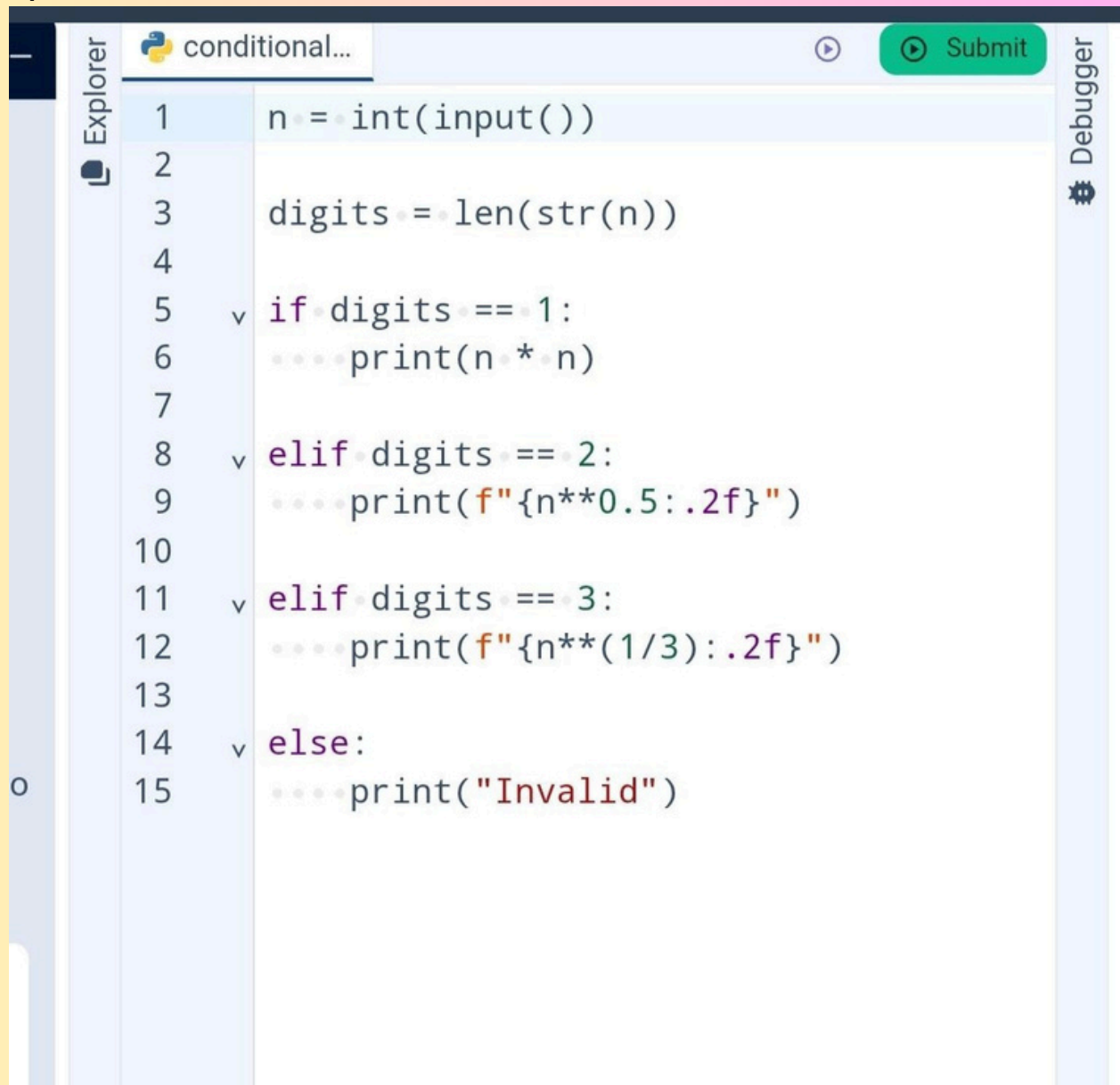
Output Format:

- If n is a single-digit number, print its square.
- If n is a two-digit number, print its square root (rounded to two decimal places).
- If n is a three-digit number, print its cube root (rounded to two decimal places).
- Else print "Invalid".

Sample Test Cases

+

Input:



The screenshot shows a code editor with a file named 'conditional...'. The code is a Python script that takes an input number and calculates its square root or cube root based on the number of digits. The code is as follows:

```
1 n = int(input())
2
3 digits = len(str(n))
4
5 if digits == 1:
6     print(n * n)
7
8 elif digits == 2:
9     print(f"{n**0.5:.2f}")
10
11 elif digits == 3:
12     print(f"{n**(1/3):.2f}")
13
14 else:
15     print("Invalid")
```

The editor has a 'Submit' button in the top right corner and a 'Debugger' icon on the right side. The left side shows a file explorer with a folder icon.

Output:

Average time

0.017 s

16.86 ms



Maximum time

0.030 s

30.00 ms



✓ 4 out of 4 shown test case(s) passed

✓ 3 out of 3 hidden test case(s) passed

✓ Test case 1 30 ms

🐞 Debug



Expected output

Actual output

9

9

81 ↵

81 ↵



< Prev

Reset

Submit

Next >

Write a Python program to calculate number of days between two dates.

Input Format:

- The first line contains the first date in the format *YYYY – MM – DD*.
- The second line contains the second date in the format *YYYY – MM – DD*.

Output Format:

- An integer representing the number of days between the given dates.

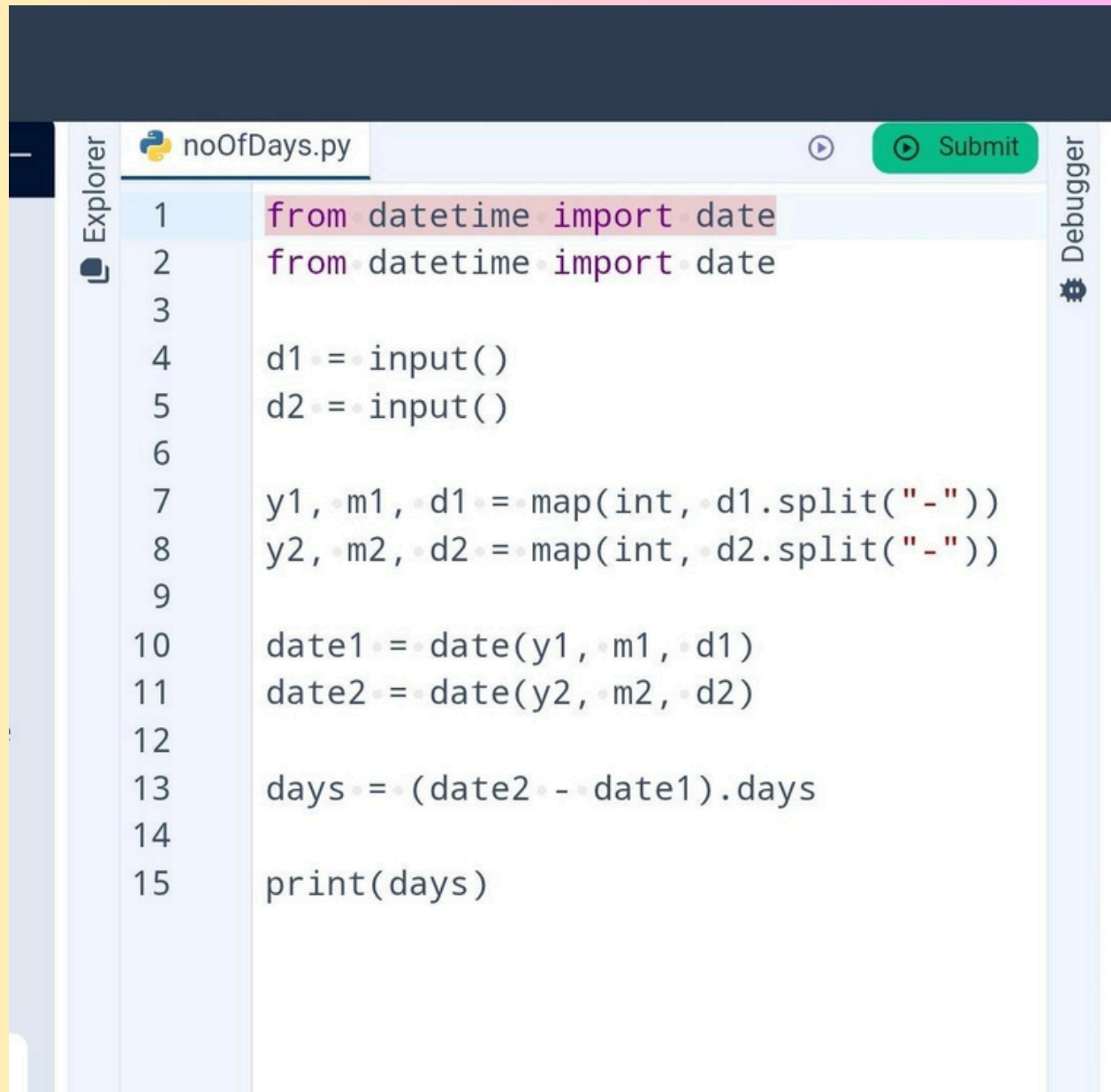
Note:

- The first date should always be considered earlier than the second date.

Sample Test Cases

+

Input:



The image shows a Python IDE window titled "noOfDays.py". The code is as follows:

```
1 from datetime import date
2 from datetime import date
3
4 d1 = input()
5 d2 = input()
6
7 y1, m1, d1 = map(int, d1.split("-"))
8 y2, m2, d2 = map(int, d2.split("-"))
9
10 date1 = date(y1, m1, d1)
11 date2 = date(y2, m2, d2)
12
13 days = (date2 - date1).days
14
15 print(days)
```

The IDE interface includes an "Explorer" panel on the left, a "Submit" button in the top right, and a "Debugger" panel on the right.

Output:

Average time

0.024 s

23.50 ms



Maximum time

0.027 s

27.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 22 ms

🐞 Debug



Expected output

Actual output

2014-07-02

2014-07-02

2014-11-02

2014-11-02

123 📄

123 📄



< Prev

Reset

Submit

Next >

1.1.4. Reverse a Number

01:15



Write a Python program to reverse the digits of the number and print the reversed number.

Input Format:

- The input is a single integer.

Output Format:

- The output prints a single integer, which is the reversed number.

Sample Test Cases



Input:

reverseNu...

Submit

Explorer

1

n = int(input())

2

3

rev = 0

4

while n > 0:

5

digit = n % 10

6

rev = rev * 10 + digit

7

n = n // 10

8

9

print(rev)

Debugger

Output:

The screenshot displays the test results for a coding problem. At the top, performance metrics are shown: Average time is 0.022 s (21.60 ms) and Maximum time is 0.043 s (43.00 ms). Below this, the test case status is reported: 2 out of 2 shown test case(s) passed, and 3 out of 3 hidden test case(s) passed. The detailed view for Test case 1 shows a duration of 43 ms and a 'Debug' button. A comparison table follows, showing that the expected output (5367 and 7635) matches the actual output (5367 and 7635). The interface includes a bottom navigation bar with buttons for '< Prev', 'Reset', 'Submit', and 'Next >'.

Performance Metrics	
Average time	Maximum time
0.022 s	0.043 s
21.60 ms	43.00 ms

Test Case Status
2 out of 2 shown test case(s) passed
3 out of 3 hidden test case(s) passed

Test case 1 (43 ms)	
Expected output	Actual output
5367	5367
7635	7635

Navigation
< Prev Reset Submit Next >

Write a Python program that takes an integer as input and prints the multiplication table for that integer from 1 to 10.

Input Format:

- The first line of input contains an integer that represents the number for which the multiplication table is to be printed.

Output Format:

- Print the multiplication table for the given number in the format:

```
<number> x <multiplier> = <result>
```

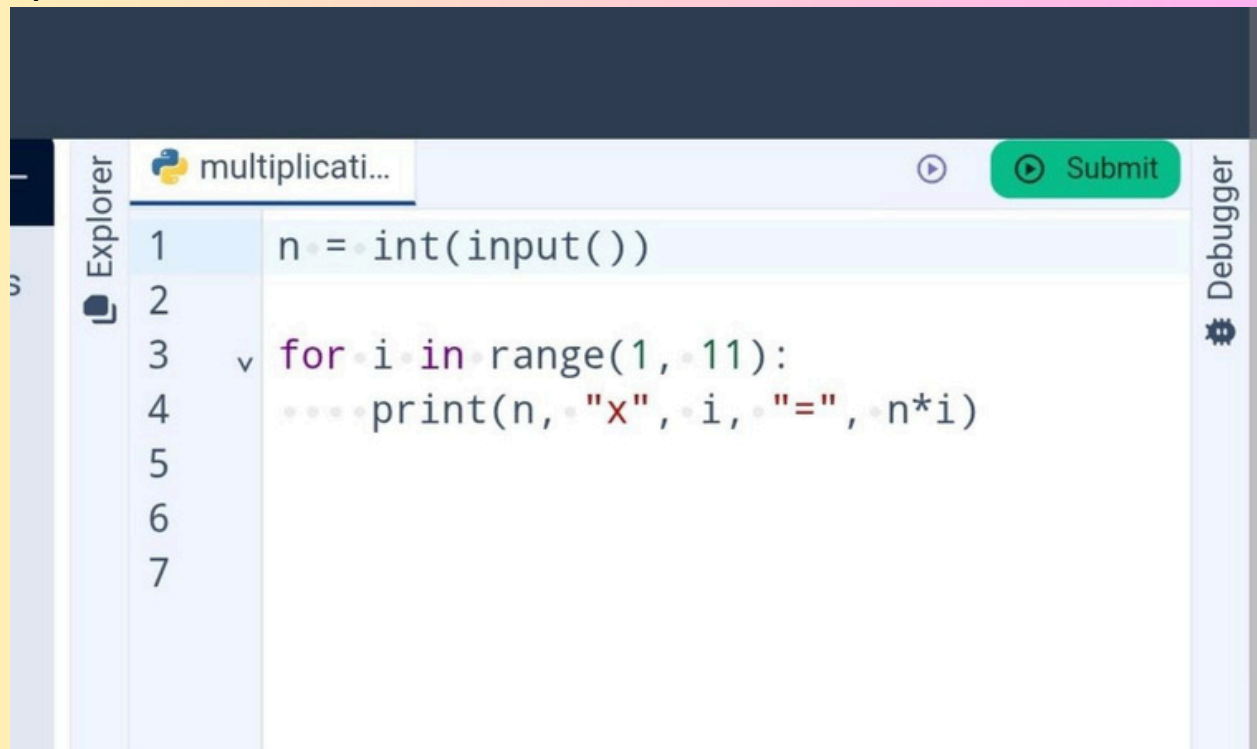
Note:

- The character "x" is used in the output to represent multiplication.
- Refer to the visible test-cases for more clarity.

Sample Test Cases

+

Input:



The screenshot shows a Python IDE interface. At the top, there's a dark blue header bar. Below it, a tab labeled 'multiplicati...' is active. To the right of the tab are a play button icon and a green 'Submit' button. On the left side, there's a vertical sidebar with the word 'Explorer' and a folder icon. The main area displays a Python script with line numbers 1 through 7 on the left. The code is as follows:

```
1 n = int(input())
2
3 for i in range(1, 11):
4     print(n, "x", i, "=", n*i)
5
6
7
```

On the right side of the code editor, there's a vertical sidebar with the word 'Debugger' and a bug icon.

Output:

Average time

0.017 s

16.50 ms



Maximum time

0.020 s

20.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 20 ms

Debug



Expected output

Actual output

8

8

$8 \cdot x \cdot 1 = 8$

$8 \cdot x \cdot 1 = 8$

$8 \cdot x \cdot 2 = 16$

$8 \cdot x \cdot 2 = 16$



< Prev

Reset

Submit

Next >

Write a Python program that accepts the number of courses and the marks of a student in those courses.

The grade is determined based on the aggregate percentage:

- If the aggregate percentage is greater than 75, the grade is Distinction.
- If the aggregate percentage is greater than or equal to 60 but less than 75, the grade is First Division.
- If the aggregate percentage is greater than or equal to 50 but less than 60, the grade is Second Division.
- If the aggregate percentage is greater than or equal to 40 but less than 50, the grade is Third Division.

Input Format:

- The first input will be an integer n , the number of courses.
- The second input will be n integers representing the marks of the student in each of the n courses, separated by a space.

Output Format:

If the student passes all courses:

- Print the aggregate percentage (formatted to two decimal places).
- Print the grade based on the aggregate percentage.

If the student fails any course (marks < 40 in any course), print:

- "Fail".

Note:

- Aggregate Percentage: Average performance across all courses, calculated by summing all marks and dividing

Input:

Explorer

passorFail....



Submit

Debugger

```
1  def cal(marks, total_courses):
2      if any(mark < 40 for mark in
   marks):
3          return "Fail"
4      total_marks = sum(marks)
5      aggregate_percentage =
   (total_marks / (total_courses *
   100)) * 100
6
7      if aggregate_percentage > 75:
8          grade = "Distinction"
9      elif aggregate_percentage >= 60:
10         grade = "First Division"
11     elif aggregate_percentage >= 50:
12         grade = "Second Division"
13     elif aggregate_percentage >= 40:
14         grade = "Third Division"
15
16     return (aggregate_percentage,
   grade)
17
18     num_courses = int(input())
19     marks = list(map(int,
   input().split()))
20
21     result = cal(marks, num_courses)
22
23     if result == "Fail":
24         print("Fail")
25     else:
26         aggregate_percentage, grade =
```

Output:

Average time

0.026 s

25.70 ms



Maximum time

0.032 s

32.00 ms



✓ 5 out of 5 shown test case(s) passed

✓ 5 out of 5 hidden test case(s) passed

✓ Test case 1 32 ms

Debug



Expected output

Actual output

4

4

55 65 78 89

55 65 78 89

Aggregate Percentage: 71.75

Aggregate Percentage: 71.75



< Prev

Reset

Submit

Next >



1.2.2. Fibonacci Series

38:34



Write a Python program that uses recursion to print the first n terms of the Fibonacci series.

Input Format:

- A single integer n representing the number of terms to generate.

Output Format:

- A single line containing the first n terms of the Fibonacci sequence, separated by spaces.

Sample Test Cases



Input:

fibonacci.py

Submit

Explorer

Debugger

```
1  def fibonacci(n):
2
3      # Standard Recursive Case
4      if n == 1:
5          return 0
6      elif n == 2:
7          return 1
8      else:
9          return fibonacci(n-1) +
          fibonacci(n-2)
10
11  n = int(input())
12  for i in range(1, n + 1):
13      print(fibonacci(i), end=" ")
14
```

Output:

Average time

0.039 s

39.25 ms



Maximum time

0.097 s

97.00 ms



2 out of 2 shown test case(s) passed



2 out of 2 hidden test case(s) passed



Test case 1

14 ms



Debug



Expected output

Actual output

5

5

0 · 1 · 1 · 2 · 3 ·

0 · 1 · 1 · 2 · 3 ·



< Prev

Reset

Submit

Next >

Write a Python program to print a pattern of asterisks in the form of a right-angled triangle.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format

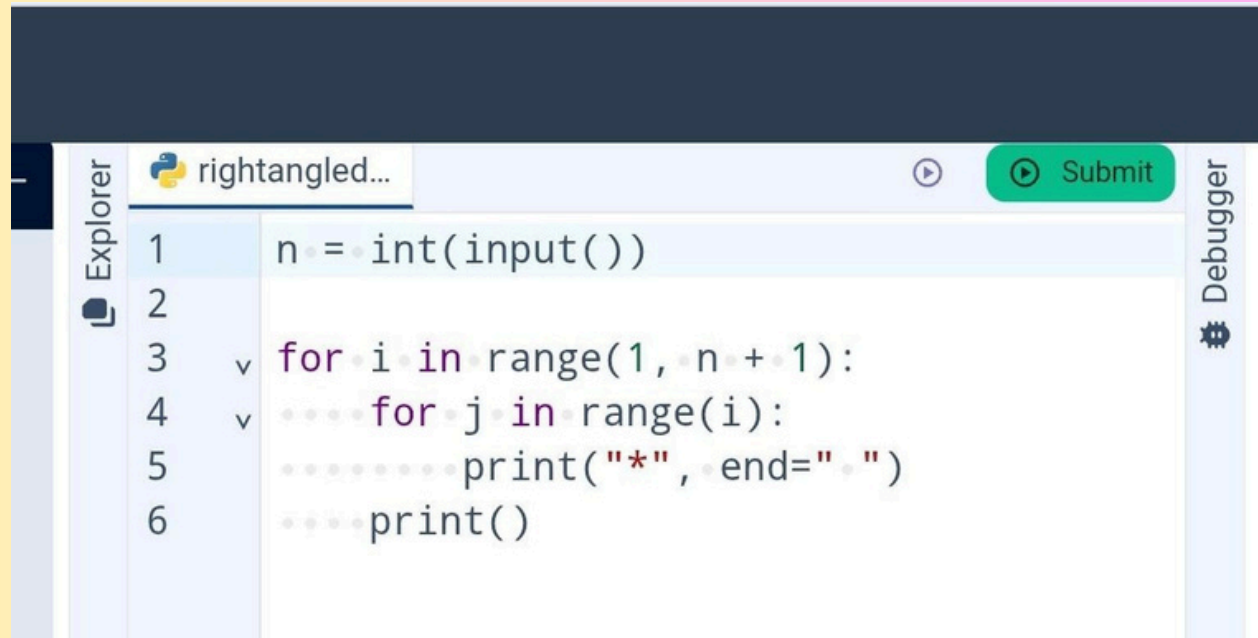
- The output should display the pattern of asterisks (*), with each row containing an increasing number of asterisks.

Note: Refer to the displayed test cases for the sample pattern.

Sample Test Cases

+

Input:



The image shows a code editor interface with a dark blue header bar. Below the header, there's a tab labeled "rightangled..." with a Python icon. To the right of the tab are a play button icon and a green "Submit" button. The editor area has a light blue background and contains the following Python code:

```
1 n = int(input())
2
3 for i in range(1, n + 1):
4     for j in range(i):
5         print("*", end=" ")
6     print()
```

On the left side of the editor, there's a vertical bar with the word "Explorer" and a folder icon. On the right side, there's a vertical bar with the word "Debugger" and a bug icon.

Output:

Average time
0.018 s
18.17 ms



Maximum time
0.022 s
22.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 4 out of 4 hidden test case(s) passed

✓ Test case 1 19 ms

Debug

⋮

☰

^

Expected output

Actual output

5

5

* .

* .

* . *

* . *

* . * .

* . * .

* . * . *

* . * . *

+ + + +

+ + + +

>-

☰

< Prev

Reset

Submit

Next >

Write a Python program to print a right-angled triangle pattern of numbers.

Input Format:

- The input is an integer, representing the number of rows in the pattern.

Output Format:

- The output should display the pattern of numbers separated by space, with each row containing increasing numbers starting from 1 up to the row number.

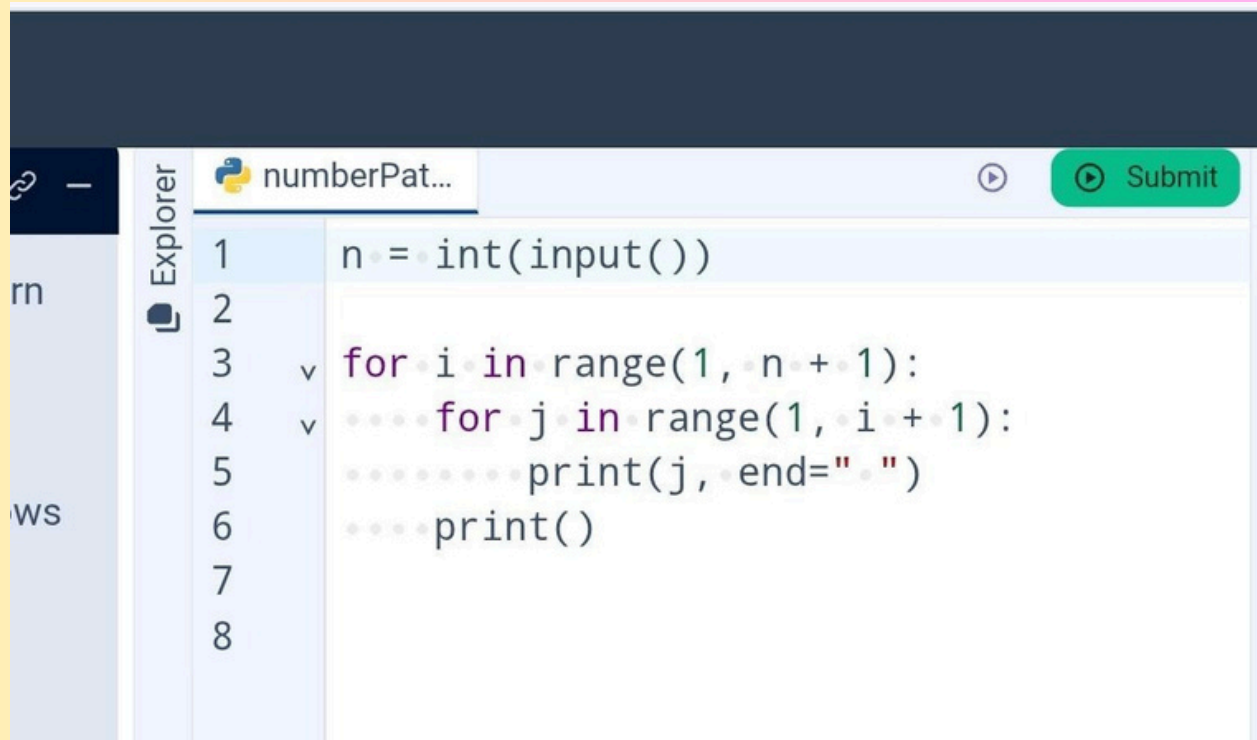
Note:

- Refer to the displayed test cases for the sample pattern.

Sample Test Cases



Input:



The screenshot shows a code editor with a dark theme. The top bar has a Python logo, the filename "numberPat...", a play button, and a green "Submit" button. The left sidebar shows a file explorer with "rn" and "WS" folders. The main editor area contains the following Python code:

```
1 n = int(input())
2
3 for i in range(1, n + 1):
4     for j in range(1, i + 1):
5         print(j, end=" ")
6     print()
7
8
```

Output:

Average time

0.020 s

19.75 ms



Maximum time

0.026 s

26.00 ms



✓ 2 out of 2 shown test case(s) passed

✓ 2 out of 2 hidden test case(s) passed

✓ Test case 1 26 ms



Debug



Expected output

Actual output

5

5

1 · ↵

1 · ↵

1 · 2 · ↵

1 · 2 · ↵

1 · 2 · 3 · ↵

1 · 2 · 3 · ↵



< Prev

Reset

Submit

Next